

Function Object-Based Overloading of Operator Dot

Doc. No.:	P0060R0
Date:	2015-09-18
Project:	Programming Language C++, Evolution Working Group
Reply To:	Mathias Gaunard < mgaunard@bloomberg.net >, Dietmar Kühl < dkuhl@bloomberg.net >

Table of Contents:

- [Introduction](#)
- [Changelog](#)
 - [P0060R0 - 2015-09-18](#)
 - [N4495 - 2015-05-22](#)
- [Motivation](#)
- [Design and Specification](#)
 - [Lookup rules](#)
 - [Member functions and scope capture](#)
 - [Extension to reflection](#)
- [Properties of the Design](#)
 - [Controlling reference leaking](#)
 - [Overloading operator.\(\) on multiple objects](#)
 - [SFINAE on the synthesized function object](#)
 - [No handling of static members](#)
- [Examples and Use Cases](#)
 - [Polymorphic Value Types](#)
 - [Sum Types](#)
 - [Dynamic Duck Typing](#)
 - [Expression Templates](#)
 - [Dynamic Properties \(reflection extension\)](#)
 - [Remote Procedure Calls with automatic marshalling \(reflection extension\)](#)
 - [Vector of values](#)
 - [Asynchronous proxy](#)
- [Acknowledgements](#)

Introduction

This proposal suggests an approach to allow overloading the dot operator that is an alternative to the solution proposed in [N4477](#). It is based on **having the compiler synthesize a function object** whenever a call to that operator is made rather than return a user-defined reference.

This approach can do everything that [N4477](#) can while avoiding some of its issues. The approach also allows some novel uses which are described below.

A simulated implementation as well as the various use cases presented in the proposal are also available as source code on github: <https://github.com/mgaunard/operator-dot>.

Changelog

P0060R0 - 2015-09-18

- Changed arguments to member functions to be passed to the `operator.` overload and then forwarded to the synthesized function object instead of constructing a stateful function object, which simplifies the design and also makes it more flexible.
- Added RPC with marshalling use case example
- Added asynchronous proxy use case example

N4495 - 2015-05-22

- Initial version

Motivation

The advent of smart pointers, types making use of the capacity to overload the arrow (`->`) operator to mimic pointers while providing extra behaviour like automatic lifetime management, has only highlighted the lack of having the ability to overload the dot (`.`) operator to achieve smart references, or better yet, smart objects.

In the following listing, we will use the terminology *smart reference* to refer to a use case that is possible with both this proposal and [N4477](#), and *smart object* for a use case that requires the method presented in this proposal.

Examples of *smart references* include:

- **Smart pointers that are really references:** smart pointer-like objects that have reference semantics rather than pointer semantics.
- **Proxies:** objects that must intercept every call to a member before rerouting it to some other object.
- **Interface refinement:** build new objects by composing existing objects, forwarding to them, and adding new members.
- **Pimpl and handles:** making an indirection to another private object transparent without having to forward every member value or function.

Examples of *smart objects* include:

- **Sum types aka variants:** objects that can hold values of different types. It is possible to define operator dot so that it forwards to the members of the actual type contained.
- **Duck typing:** similar to sum types, but the error of not having a particular member function for a given type is delayed to when the binding to the actual value occurs.
- **Expression templates and DSEs:** building a type that allows delayed evaluation by building an object representing the call rather than evaluating it is very useful to build *Domain-Specific Embedded Languages* that are based on the famous expression templates technique.
- **Aggregates of key-value pairs:** objects that are aggregates of runtime-defined properties could be accessed through operator dot instead of an explicit string.
- **Remote Procedure Calls:** an object with member functions that transparently get translated to a request/response messaging system to call the function on a remote or distributed service.
- **Repeated evaluation:** objects that do not forward the member access to a single object but to several objects in succession.
- **Asynchronous objects:** object proxies which transform each member function call into an asynchronous call returning a future.

Additionally, the function object approach allows fine control of what is forwarded, under what condition, and in what order. The discussion below provides more details.

Design and Specification

Lookup rules

When looking up a non-static member of a user-defined-type, if no member by the name requested exists, and the type has a declaration of a non-static member function named `operator.` with matching cv-qualifiers, then the compiler shall synthesize a function object with a `operator()` member function template taking an arbitrary object as parameter and applying the name to it. An instance of that synthesized function object is passed as argument to the `operator.` member function.

For example, `x.some_name;` gets translated into `x.operator.(synthesized_function_type{})`; where `x.some_name` wouldn't otherwise be found.

In this case, the synthesized function type could be equivalent to the following:

```
struct synthesized_function_type {
    template<class T>
    auto operator()(T&& t) -> decltype(t.some_name) noexcept(t.some_name) {
        return t.some_name;
    }
};
```

Special capture behavior for calls to member functions is described in the next section.

Member functions and scope capture

If the compiler must synthesize a function object for a member function call, each subexpression evaluating each argument shall be passed along as a template-parameter-pack to the `operator.` overload, which may then forward those arguments to the synthesized function object.

For example, the calls `x.some_name(a, foo(), foo())` shall get translated to to

`x.operator.(synthesized_function_type(), a, foo(), foo());` where the synthesized function type could be equivalent to the following:

```
struct synthesized_function_type {
    template<class T, class... Args>
    auto operator()(T&& t, Args&&... args) -> decltype(t.some_name(std::forward<Args>(args)...)) noexcept(t.some_name)
    {
        return t.some_name(std::forward<Args>(args)...);
    }
};
```

Extension to reflection

The mechanism could optionally be extended to support **reflection** by adding arbitrary extra information within the synthesized function object type.

Further work will be done in that area in future versions of the proposal depending on feedback.

For example for member variables:

```
struct synthesized_function_type {
    template<class T>
    auto operator()(T&& t) -> decltype((t.some_name)) noexcept((t.some_name)) {
        return t.some_name;
    }

    static constexpr const char* name() { return "some_name"; }
    static constexpr bool is_member_function = false;
};
```

For member functions:

```
struct synthesized_function_type {
    template<class T, class... Args>
    auto operator()(T&& t, Args&&... args) -> decltype(t.some_name(std::forward<Args>(args)...)) noexcept(t.some_name)
    {
        return t.some_name(std::forward<Args>(args)...);
    }

    static constexpr const char* name() { return "some_name"; }
    static constexpr bool is_member_function = true;
};
```

Properties of the Design

Controlling reference leaking

[N4477](#) discusses *reference leaking* in section 5: the possibility that `operator.()` can result in functions accidentally returning a reference to an object held by a smart reference. Since N4477 proposes the implementation of `operator.()` in similar ways as `operator->()` but returning a reference rather than a pointer the control of the returned reference needs to be in the language.

When implementing `operator.()` in terms of a passed function object the implementer of a smart reference has full control over the type returned from `operator.()`. In its simplest form reference leaking can be prevented by not returning anything from the resulting operator:

```
class strictly_non_leaking {
    X x;
public:
    template<class Fun, class... Args>
    auto operator.(Fun&& fun, Args&&... args) -> void { fun(x, std::forward<Args>(args)...); }
};
```

While that may be viable for some smart reference types, most smart reference would probably want to return a suitable result. It would be straight forward to prevent certain known return types, e.g., by conditionally wrapping results into a suitable type:

```
class wrapping {
    X x;
public:
    wrapping(X& ref);
    template<class Fun, class... Args>
    auto operator.(Fun&& fun, Args&&... args)
        -> std::condition_t<std::is_same_v<X&, decltype(fun(std::declval<X&>(), std::forward<Args>(args)...))>,
            wrapping, decltype(fun(std::declval<X&>(), std::forward<Args>(args)...))> {
        return fun(x, std::forward<Args>(args)...);
    }
};
```

What exactly a smart reference returns from a use can be controlled by the smart reference. In particular the good (`incr(x)`) case can be supported while the bad (`leak(x)`) case can be banned.

Overloading operator.() on multiple objects

N4477 discusses overloading `operator.()` in section 4.9. While there it is obvious that `operator.()` can be overloaded for cv-qualified version and reference qualifications, N4477 also proposes to overload on the reference type returned from `operator.()`. The idea is that multiple version of `operator.()` can be used to return different reference types and the unique version applicable for a member use is chosen. The choice of `operator.()` based on the reference type is similar to the choice made when finding a member function in a class with multiple base classes: if a unique match is found it is chosen otherwise (if there are no or multiple matches) the use is an error.

When passing a function object to `operator.()` this form of overloading isn't possible. However, such special overloading rules are not needed as the same effect can be achieved by determining if a function call can be made:

```
class composite {
    A a;
    B b;
public:
    template class Fun, class... Args>
    auto operator.(Fun&& fun, Args&&... args)
        -> decltype(call_unique(std::bind(std::forward<Fun>(fun), std::forward<Args>(args)...), std::tie(this->a, 1
        return call_unique(std::bind(std::forward<Fun>(fun), std::forward<Args>(args)...), std::tie(this->a, this->
    }
};
```

The function `call_unique()` determines if there is exactly one element `x` in the passed `std::tuple<...>` for which `fun(x)` is a valid called and, if so, returns the result of calling `fun` with this element. Otherwise, it produces an error. This function isn't easy to write but it could be made available by the standard library to aid with common choices. Since it is a library function other choices could be made, however. For example, instead of calling the unique choice a different approach could be to call the first match.

SFINAE on the synthesized function object

A lot of advanced uses of overloading `operator.` with the scheme described on this proposal rely on SFINAE extended for expressions. SFINAE for expressions is necessary to be able to tell whether a member exists for a particular object, i.e., whether the synthesized function object can be called with a specific parameter type.

This is why the signature of the `operator.()` member function template of the synthesized function objects has a return type defined with `decltype` rather than just relying on a specification using `auto` or `decltype(auto)`.

No handling of static members

To build perfect smart references or proxies, it would not only be required to forward regular members, but static ones as well. For example, consider a proxy for `std::vector` which should have the `::iterator` be provided, too.

Like N4477, the proposal currently doesn't cover static members. The mechanism could be extended to support these members. The main issue is being able to deal with both types and values, in particular in contexts where member types are not annotated with `typename`.

Examples and Use Cases

Polymorphic Value Types

Inheritance and subtyping polymorphism is a use case for some sort of dynamic typing. If given a base class `Base`, and several derived classes `Derived1`, `Derived2` or `Derivedn`, it is often useful to be able to have an object that can contain any of the `Derivedi` classes derived from `Base`.

Smart pointers are a popular solutions to this problem. While smart pointers provide an entity semantic, there is also an argument that can be made for having value semantics instead, where copies actually copies the data instead of aliasing it.

This copying can be achieved using a special smart pointer. There are several implementations of this approach on the Internet under the name `clone_ptr`. Pointer syntax is however not very appropriate when providing something using value semantics. Here is an example of what it could look like with this proposal. Let's name the type that can hold any type derived from `T` while providing a value semantic `poly<T>`, below is a simplistic implementation relying on an intrusive "clone" virtual member function and lacking move semantics and other fancy features.

```
template<class T>
struct poly {
    poly(T const& t) : ptr(new T(t)) {}
    ~poly() { delete ptr; }

    poly(const poly& other) : ptr(other.p->clone()) {}

    poly& operator=(poly other) {
        std::swap(ptr, other.ptr);
        return *this;
    }

    template<class F, class... Args>
    auto operator.(F f, Args&&... args) -> decltype(f(std::declval<T&>(), std::forward<Args>(args)...))
        return f(*ptr, std::forward<Args>(args)...);

    template<class F, class... Args>
    auto operator.(F f, Args&&... args) const -> decltype(f(std::declval<T const&>(), std::forward<Args>(args)...))
        return f(static_cast<T const&>(*ptr), std::forward<Args>(args)...);

private:
    T* ptr;
};
```

It then becomes possible to write things like this:

```
poly<Base> obj = Derived1();
a.virtual_member_function_of_Base();
a = Derived2();
int x = a.member_variable_of_Base;
```

Sum Types

Sum types, also called tagged unions or variants, are data structures that can hold one object out of a list of possibly unrelated types at a time. Proposal [N4450](#) suggests adding such a type to the standard library, named `variant`, based on the `boost::variant` class template.

Proposal [N4450](#) for the variant type provides very limited operator overloading (only `<`, `<=`, `=`, `!=`, `>=` and `>`), but a case could be made for providing overloading for all operators, including operator dot. This raises a couple of interesting questions regarding what the result type of that operator should be, and whether a hard error should be emitted if the operator in question is not available on one or more of the types in the set.

For the use case below, we present a partial interface of a simple variant implementation with freestore-based storage and pseudo-code to ignore some of the implementation complications inherent to variant. Operator dot returns the common type of all possible cases, and the operator being called must be valid for all cases.

```
template<class... T>
struct variant {
    template<class U>
    variant(U const& u) : ptr(new U(u)), which(find_offset<seq<T...>, U>::value) {}

    ~variant() { type_erased_delete(ptr); }
```

```

variant(variant const& other) : ptr(type_erased_clone(other)), which(other.which) {}

variant& operator=(variant other) {
    swap(ptr, other.ptr);
    swap(which, other.which);
    return *this;
}

// collapse_result_of ensures the overload is only valid if all cases return the same type
template<class F, class... Args>
auto operator.(F f, Args&&... args) -> typename collapse_result_of<F(Args...), T...>::type {
    switch(which)
    {
        case 0: return f(*static_cast<T0*>(ptr), std::forward<Args>(args)...);
        case 1: return f(*static_cast<T1*>(ptr), std::forward<Args>(args)...);
        /* for every Ti... */
    }
}

template<class F, class... Args>
auto operator.(F f, Args&&... args) const -> -> typename collapse_result_of<F(Args...), T...>::type {
    switch(which)
    {
        case 0: return f(*static_cast<T0 const*>(ptr), std::forward<Args>(args)...);
        case 1: return f(*static_cast<T1 const*>(ptr), std::forward<Args>(args)...);
        /* for every Ti... */
    }
}

private:
    void* ptr;
    int which;
};

```

With the `collapse_result_of` meta-function defined as such:

```

template<class Sig, class... Types>
struct collapse_result_of;

template<class F, class... Args, class Type>
struct collapse_result_of<F(Args...), Type>
    : std::result_of<F(Type, Args...)>
{};

template<class F, class... Args, class Type, class... Types>
struct collapse_result_of<F(Args...), Type, Types...>
    : collapse_result_of<F(Args...), Types...>
    , std::result_of<F(Type, Args...)>
{};

```

This use case is a prime example of a smart object. This interface can be achieved by synthesizing a function object but it cannot be achieved by forwarding to a reference like in N4477 (operator dot), since there is no reference to a single object that variant could return.

It becomes possible to write things like this:

```

struct Foo { const char* name() { return "Foo"; } };
struct Bar { const char* name() { return "Bar"; } void bark() { cout << name() << endl; } };

variant<Foo, Bar> v = Foo();
v = Bar();
const char* s = v.name();
//v.bark(); // error: not all types define 'bark'

```

Dynamic Duck Typing

Duck typing is a technique which involves binding a name to an object as late as possible: if the name is available at the time it is needed, call it, otherwise raise an error. Most dynamic typed language are based on this principle as it provides a very easy and flexible programming model.

It cannot be implemented in C++ generally, but it is possible to provide duck typing over a finite set of types, so it is possible to provide it for a variant type like above. It requires being able to test at compile-time if a given type satisfies the call, in order to be able to fallback to code

that generates an error in case the expression is not supported.

This is one use case that requires the synthesized function object to contain its full body in its signature, so that it can be used in SFINAE contexts.

From a synthesized function object for the operator dot call, the implementation would wrap it in another function object with a fallback by doing something like this:

```
template<class T, class R = void>
struct sink { typedef R type; };

template<class Sig, class Enable = void>
struct is_callable : std::false_type {};

template<class F, class... Args>
struct is_callable<F(Args...), typename sink<decltype(std::declval<F>()(std::declval<Args>()...))>::type> : std::true_type {};

template<class F, class R>
struct call_or_throw : F {
    using F::operator();

    template<class T, class... Args>
    typename std::enable_if<!is_callable<F(T&&, Args&&...)>::value, R>::type operator()(T&& t, Args&&...) const
    {
        throw std::runtime_error("No such operation");
    }
};
```

The `operator.` overload now looks like this:

```
template<class... T>
struct duck_variant {
    /* content from variant... */

    // require all valid cases to return the same type
    template<class F, class... Args>
    auto operator.(F f, Args&&... args) -> typename collapse_result_of<F(Args&&...), T...>::type {

        typedef typename collapse_result_of<F(Args&&...), T...>::type R;

        switch(which)
        {
            case 0: return call_or_throw<F, R>{f}(*static_cast<T0*>(ptr), std::forward<Args>(args)...);
            case 1: return call_or_throw<F, R>{f}(*static_cast<T1*>(ptr), std::forward<Args>(args)...);
            /* for every Ti... */
        }
    }
};
```

It becomes possible to write things like this:

```
struct Foo { const char* name() { return "Foo"; } };
struct Bar { const char* name() { return "Bar"; } void bark() { cout << name() << endl; } };

duck_variant<Foo, Bar> v = Foo();
v = Bar();
const char* s = v.name();
v.bark(); // correct even if Foo has no 'bark'
```

Expression Templates

Expression templates is a mechanism using operator overloading to delay evaluations of expressions and then evaluate them with a given context, which enables building entire Domain-Specific Languages embedded into C++. For this use case, forwarding to a reference is not possible, and generating a function object fits exactly the needs of delaying evaluation that is needed.

In this case, the `operator.` would just return the synthesized function object or more likely a wrapper of that function object.

Dynamic Properties (reflection extension)

Another use case where overloading the dot operator is useful is for objects where the member names are dynamic, like those obtained from binding to a dynamic language, serialization to XML or JSON, or anything deciding fields at runtime. This cannot be addressed by just providing a function object, but it is trivial to extend the function objects to contain additional information to support that use case. With N4477 (operator dot), there is no obvious way to extend the mechanism to support this use case.

Assuming the synthesized function object is augmented to support reflection, i.e., a `name()` function return the name of the member being applied, it could therefore look like this:

```
struct Value {
    Value() : data(Map()) {}
    Value(int i) : data(i) {}
    Value(double d) : data(d) {}
    Value(std::string const& s) : data(s) {}

    template<class F>
    Value& operator.(F f) {
        return get<Map>(data)[F::name()];
    }

    template<class F>
    Value const& operator.(F f) const {
        return get<Map>(data)[F::name()];
    }

private:
    typedef std::unordered_map<std::string, Value> Map;
    variant<Map, int, double, std::string> data;
}
```

And this would allow the following usage:

```
Value v;
v.foo = 42;
v.bar = "bar";
```

Adding "foo" and "bar" members dynamically.

Remote Procedure Calls with automatic marshalling (reflection extension)

With Service-Oriented Architectures and distributed systems becoming increasingly popular, it is often necessary to call a function on a remote machine, after having serialized the arguments to the function, and then deserializing the result.

The existing bindings for such frameworks in C++ are significantly more verbose than their equivalent in dynamic languages, and overloading `operator.` could allow to simplify them a lot.

```
struct my_service {
    template<class F, class... Args>
    Value operator.(F f, Args&&... args) {
        std::vector<Value> value_args = { Value(args)... };
        return send_request(F::name(), value_args);
    }

private:
    // serialize `function_name` and `args` to message
    // send request message to service
    // get response message from service
    // deserialize it to Value
    Value send_request(const char* function_name, std::vector<Value> const& args);
};
```

Then an object could be used like this:

```
news_service.latest_news("John Doe", 100);
```

which could transparently generate a JSON-RPC request "latest_news" to a news service to get the 100 latest news about John Doe.

Vector of values

Just like `+` on `std::valarray` adds all of its values, it could also be interesting to call a member on all the values of the vector as well. Consider `valarray< complex<T> >`, it would be useful to be able to apply `.real()` and `.imag()` to all members of the vector at once without requiring special specialization.

This is a use case that requires calling the function object several times on different values, and that isn't supported by forwarding it to a single reference like in [N4477](#).

```
template<class T>
struct array {
    template<class F, class... Args>
    auto operator.(F f, Args&&... args) -> array< decltype(f(values[0], std::forward<Args>(args)...)) > {
        array< decltype(f(values[0], std::forward<Args>(args)...)) > result(values.size());
        for(size_t i=0; i<values.size(); ++i)
            result[i] = f(values[i]);
        }
    return result;
}

private:
    std::vector<T> values;
};
```

which would then be used to do the following:

```
std::valarray< std::complex<float> > complex_numbers;
std::valarray< float > real_part = complex_numbers.real();
std::valarray< float > imaginary_part = complex_numbers.imag();
```

Asynchronous proxy

A normal object runs each of its function synchronously; it can however be interesting to provide an object adaptor that makes all those member functions run asynchronously by returning a future to their result instead.

Additionally, if an argument to a member function to an asynchronous object is itself a future, it can add the argument as a dependency to the new asynchronous call instead of waiting for the argument to be available.

All of this can be done generically and transparently by overloading `operator.`.

```
template<class T>
struct async_obj {
    template<class... Args>
    async_obj(Args&&...) : obj(std::forward<Args>(args)...) {}

    template<class F, class... Args>
    auto operator.(F f, Args&&... args) -> std::future< decltype(f(std::declval<T>()), std::forward<Args>(args)...) > {
        return is_future<Args...> ?
            std::when_all(wrap(obj), wrap(std::forward<Args>(args))...).then(f) :
            std::async(f, obj, std::forward<Args>(args)...);
    }

private:
    T obj;
};
```

with the following implementation details to convert arguments back and forth between futures:

```
template<class T>
T&& unwrap(T&&) { return t; }
template<class T>
T unwrap(future<T>&&) { return t.get(); }
template<class T>
using unwrap_t = decltype(unwrap(std::declval<T>()));

template<class T>
future<T> wrap(T&& t) { return std::make_ready_future<T>(t); }
template<class T>
future<T>&& wrap(future<T>&& t) { return t; }

template<class... T>
```

```
bool is_future = !std::is_same<T, unwrap_t<T> > || ... ; // fold expression
```

then given a user-defined object like this:

```
struct my_object
{
    double compute_approximation(double value);
    double compute_solution(double value, double approximation);
};
```

You could obtain something like so:

```
async_obj<my_object> obj;
std::future<double> approx = obj.compute_approximation(42.);
std::future<double> sol = obj.compute_solution(42., approx);
```

this code doesn't block and automatically builds a data flow between `compute_approximation` and `compute_solution`.

Acknowledgements

- Thanks to [Phil Miller](#) for having inspired the RPC use case.
- Thanks to [Giovanni Piero Deretta](#) for providing the asynchronous proxy use case.